

Replicating DeepFM: An End-to-End Model for CTR Prediction

A Guided Tour of the Architecture and Implementation



Project Repository: github.com/RuokeZhang/CTR_Prediction

Based on the paper: *DeepFM: A Factorization-Machine based Neural Network for CTR Prediction*

The Multi-Billion Dollar Question: Will a User Click?

What is CTR?

Click-Through Rate (CTR) is the ratio of clicks on a link to the number of times it's shown (impressions). It's a fundamental metric for measuring the success of online advertising and recommendation campaigns.

$$CTR = \frac{\text{Clicks}}{\text{Impressions}} \times 100\%$$

Why Does It Matter?

In applications with large user bases like e-commerce or advertising, even a tiny improvement in prediction accuracy (e.g., 0.1% or 0.001) can lead to a significant increase in platform revenue and user experience.

The Data Challenge

CTR prediction data is unique. It's typically:

- **Massive-Scale:** Billions of samples are common.
- **High-Dimensional & Sparse:** Features can run into the millions after one-hot encoding.
- **A Mix of Features:** Contains both dense numerical and high-cardinality categorical features.



The Foundation: A Landmark Paper and a Benchmark Dataset



The Paper – DeepFM

Full Title: DeepFM: A Factorization-Machine based Neural Network for CTR Prediction

Authors: Huifeng Guo, Ruiming Tang, Yunming Ye, Zhenguo Li, Xiuqiang He

Conference: IJCAI 2017

Core Idea: Proposes an end-to-end model that learns both low- and high-order feature interactions simultaneously, without needing manual feature engineering.



The Dataset – Criteo

Source: Released by Criteo AI Lab for the Kaggle Display Advertising Challenge (2014).

Scale: ~45 million samples.

Features: A mix of 13 numerical (dense) features and 26 categorical (sparse) features.

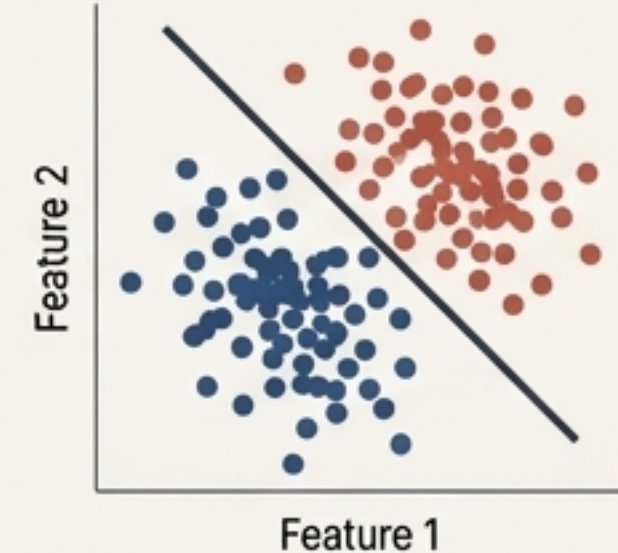
Task: Binary classification to predict if an ad was clicked. It has become the de facto public benchmark for evaluating CTR models.

This project is a hands-on implementation and replication of the DeepFM model on the Criteo dataset.

Measuring Success: Baselines and Evaluation Metrics

The Baseline: Logistic Regression (LR)

A simple, efficient, and surprisingly strong baseline. It models linear relationships but fails to capture non-linear feature interactions without extensive manual feature engineering (e.g., creating cross-product features).



The Metrics

AUC (Area Under ROC Curve)

Measures: The model's ability to rank positive instances higher than negative ones. 1.0 is perfect; 0.5 is random.

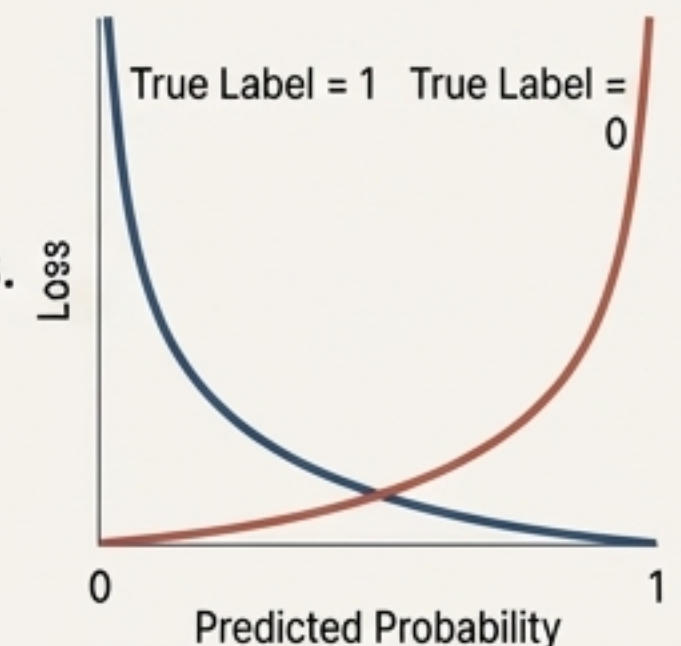
Why: A robust metric for ranking quality.



Logloss (Binary Cross-Entropy)

Measures: The penalty for incorrect predictions. Lower is better. Heavily penalizes confident but wrong predictions.

Why: Directly measures the accuracy of predicted probabilities.



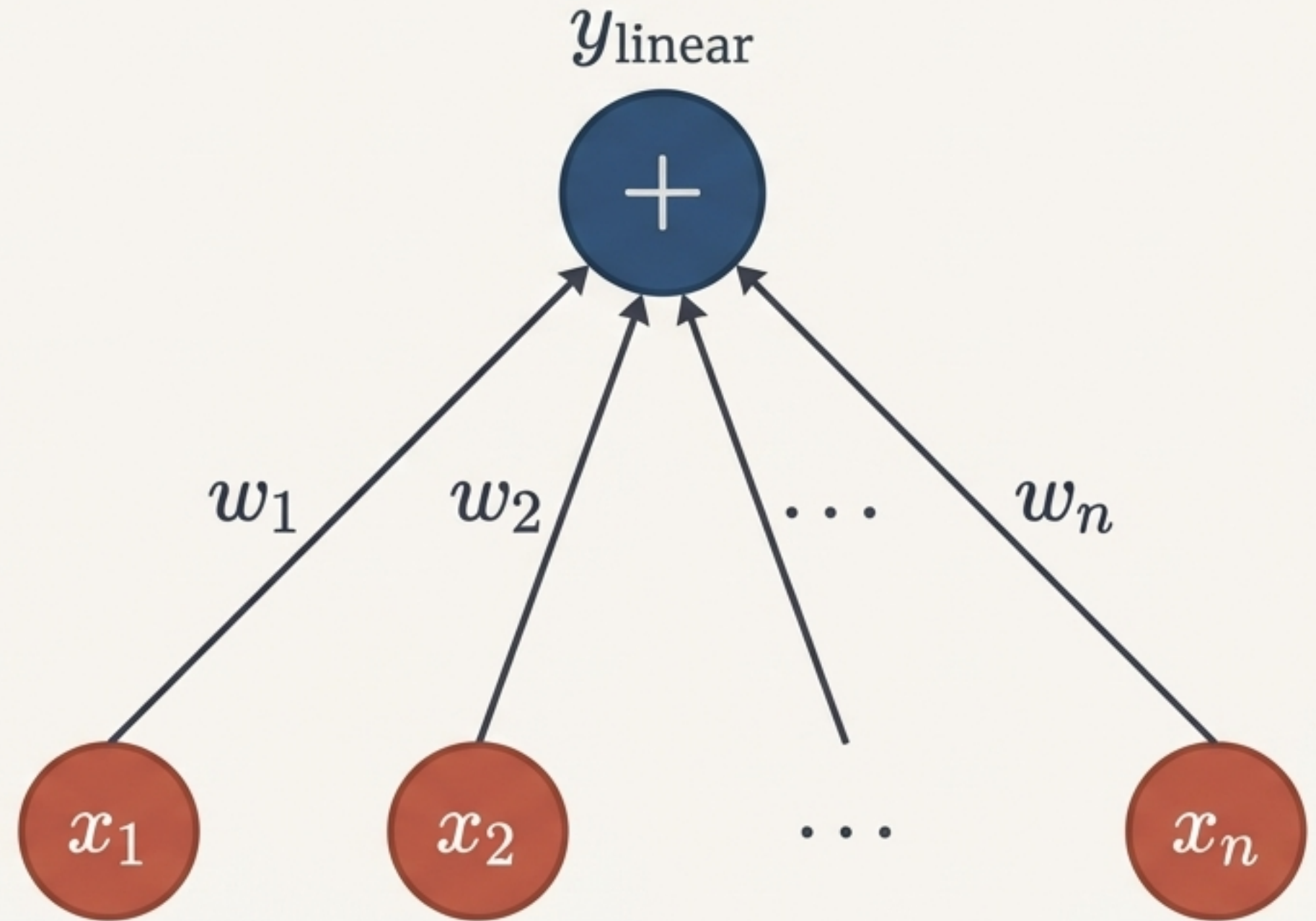
Building DeepFM, Part 1: The Linear Component for First-Order Features

Core Concept

The first step is to model the direct, linear influence of each feature. This captures the importance of individual features (e.g., 'users from this city are generally more likely to click'). This is essentially a Logistic Regression model.

Equation

$$y_{\text{linear}} = \langle w, x \rangle$$



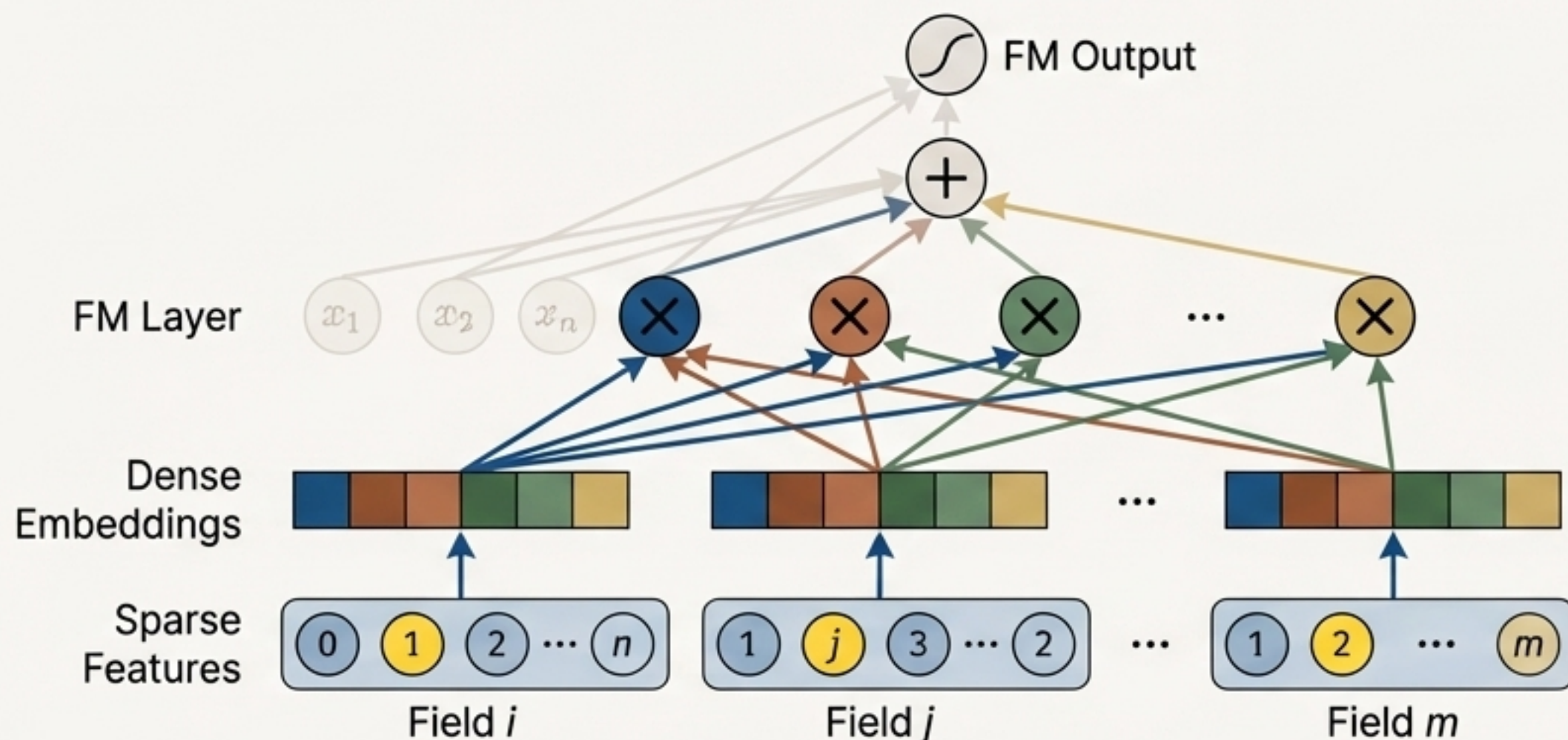
Building DeepFM, Part 2: The FM Component for Pairwise Interactions

The Limitation of Linear Models

They can't learn that the combination of two features is more predictive than their individual effects (e.g., "users who like shooting games" + "are male teenagers").

The FM Solution

Factorization Machines model pairwise feature interactions using latent vectors (embeddings). The interaction between feature i and j is the inner product of their latent vectors: $\langle V_i, V_j \rangle$.



Key Insight

FM can effectively learn interaction weights even for feature pairs that rarely appear together in the data, because the latent vectors V_i and V_j are learned from all data points containing feature i or j , respectively.

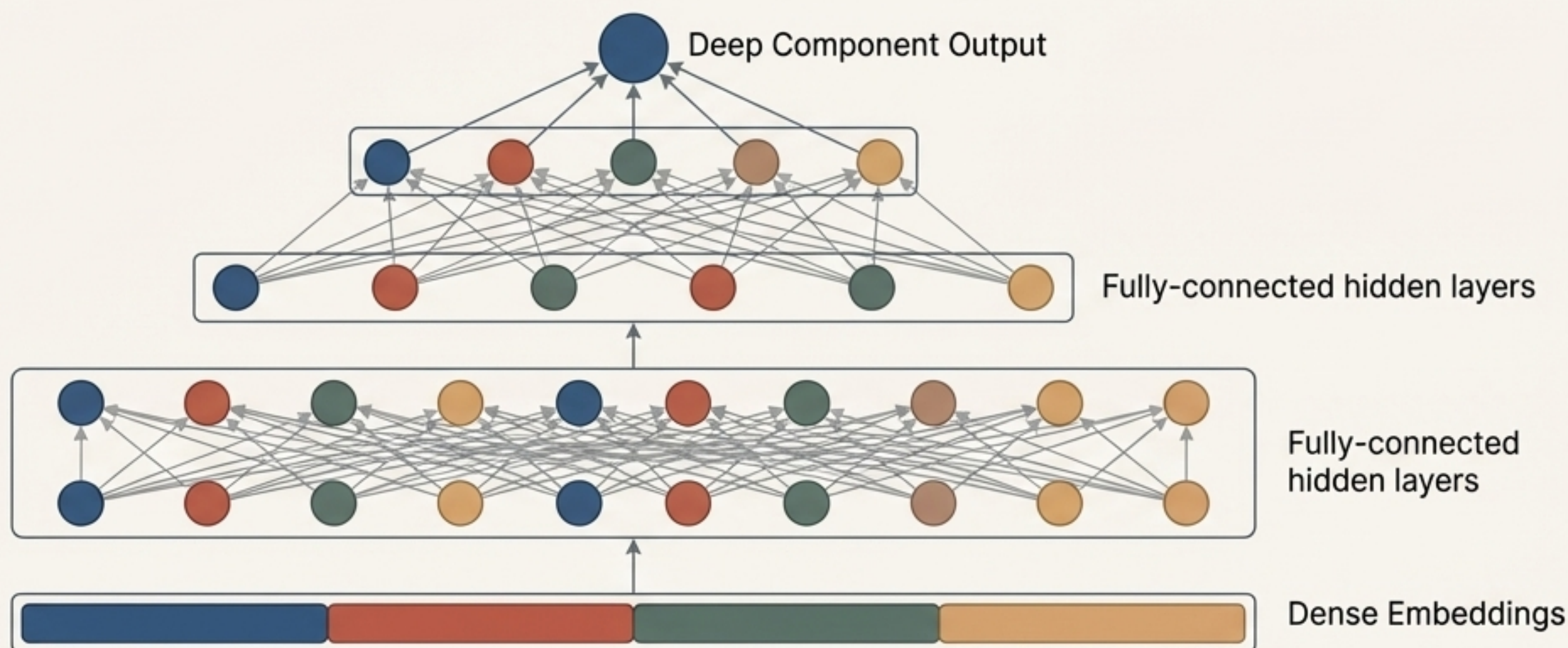
Building DeepFM, Part 3: The Deep Component for High-Order Interactions

The Next Frontier

While FM is powerful for second-order interactions (feature A + B), it struggles with higher-order combinations (e.g., “male teenagers” + “like shooting games” + “on weekends”).

The DNN Solution

A standard feed-forward neural network (MLP) can learn deep, complex, and implicit feature interactions automatically by passing dense inputs through multiple hidden layers with non-linear activations.



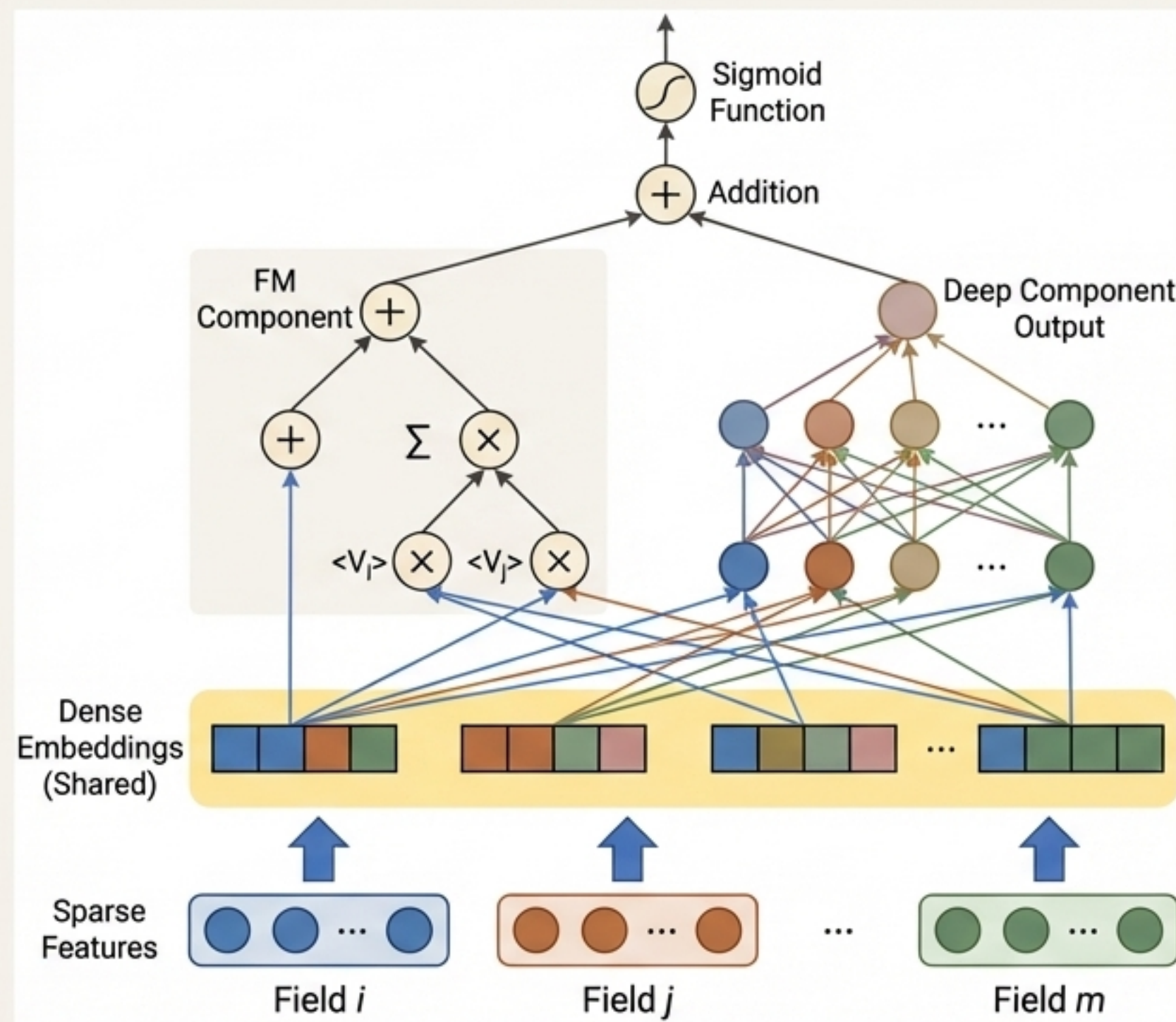
The Synthesis: Combining FM and DNN into a Single End-to-End Model

The Core Innovation

DeepFM shares the same input and, most importantly, the same feature embedding layer for both its “wide” (FM) and “deep” (DNN) components.

Key Benefits

- **No Feature Engineering:** Learns all interactions directly from raw features.
- **End-to-End Training:** The entire network is trained jointly.
- **Comprehensive:** Captures both low-order (FM) and high-order (DNN) interactions.



From Architecture to Code: A Look at the Implementation

The elegance of the DeepFM architecture translates to a modular implementation. We can build each component separately and then combine them in the forward pass.

```
# 1. Define the shared Embedding Layer
self.embedding_dict = create_embedding_matrix(...)
# 2. Define the Linear/FM part
self.linear = Linear(...)
self.fm = FMLayer()
# 3. Define the Deep part
self.dnn = DNN(layers=[256, 128])
# Forward Pass
def forward(self, x):
    sparse_embeddings = self.embedding_dict(x)
    # FM Path
    linear_out = self.linear(x)
    fm_out = self.fm(sparse_embeddings)
    # Deep Path
    dnn_input = torch.flatten(sparse_embeddings, start_dim=1)
    dnn_out = self.dnn(dnn_input)
    # Combine and predict
    final_logit = linear_out + fm_out + dnn_out
    return torch.sigmoid(final_logit)
```

The `'embedding_dict'` is the key shared component.

The `'FMLayer'` computes the pairwise inner products.

The `'DNN'` module is a standard multi-layer perceptron.

The `'forward'` method clearly shows the two parallel paths and their final combination.

DeepFM's Legacy: A Powerful and Efficient Foundation for CTR Prediction

Key Takeaways

- **A Powerful Synthesis:** Brilliantly combines the strengths of Factorization Machines (for low-order interactions) and Deep Neural Networks (for high-order interactions).
- **Efficiency and Simplicity:** By sharing the embedding layer, it avoids the complex, manual feature engineering seen in earlier models like Google's Wide & Deep.
- **Strong Performance:** A consistently high-performing model that often serves as a strong baseline for newer, more complex architectures.



The Broader Context

DeepFM paved the way for more advanced models focusing on specific aspects of feature interaction. Models like **xDeepFM** designed more explicit interaction networks, while models like **DIN (Deep Interest Network)** introduced attention mechanisms. However, DeepFM remains a crucial model to understand and a highly practical choice for many real-world systems due to its excellent balance of performance and efficiency.